Reviewer Pack — Maximum Fourier Coefficient Lower Bound via Sensitivity Scal…

Entry eff6dec1e004 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 21:18:06 UTC

Maximum Fourier Coefficient Lower Bound via Sensitivity Scaling

Entry ID: `eff6dec1e004` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-07 21:18:06 UTC

1. Conjecture

Field A (mathematical branch): Fourier Analysis of Boolean Functions **Field B** (complexity object): Decision Tree Size

Statement:

> For all boolean functions f: {0,1}^n → {0,1}, max_{S} |f̂(S)| ≥ sensitivity(f) / √n. Equality holds for parity functions.

Rationale (proposer's reasoning):

> Fourier coefficients encode variable influence, while sensitivity measures local changes. This conjecture bridges their relationship, suggesting that high sensitivity implies significant Fourier mass, which could expose structural constraints on decision tree size.

Taxonomy category: `FOURIER_ANALYTIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `639e9f7367d4c511`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def fast_walsh_hadamard_transform(f):
        n = len(f)
        if n == 1:
            return f
        even = fast_walsh_hadamard_transform([f[i] + f[i + n // 2] for i in range(n // 2)])
        odd = fast_walsh_hadamard_transform([f[i] - f[i + n // 2] for i in range(n // 2)])
        return [even[i] + odd[i] for i in range(n // 2)] + [even[i] - odd[i] for i in range(n // 2

    def sensitivity(f, n):
        max_sens = 0
        for i in range(n):
            sens = sum(abs(f[x] - f[toggle_bit(x, i)]) for x in range(1 << n) if x & (1 << i))
            max_sens = max(max_sens, sens)
        return max_sens

    def toggle_bit(x, i):
        return x ^ (1 << i)

    n = random.randint(5, 40)
    f = [random.choice([0, 1]) for _ in range(1 << n)]
    Fourier_coefficients = fast_walsh_hadamard_transform(f)
    max_Fourier_coefficient = max(abs(coeff) for coeff in Fourier_coefficients)
    sens = sensitivity(f, n)

    metric_name = "Fourier Coefficient Lower Bound"
    metric_value = max_Fourier_coefficient
    instances_tested = 1
    conjecture_holds = max_Fourier_coefficient >= sens / math.sqrt(n)
    counterexample = "" if conjecture_holds else f"n={n}, sensitivity={sens}, max_Fourier_coeffici

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
```

```
seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric_value = sum(res["metric_value"] for res in results) / len(results)
std_metric_value = math.sqrt(sum((res["metric_value"] - mean_metric_value)**2 for res in resul
support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractio
elif any(not res["conjecture_holds"] for res in results):
    first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
    print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)]['cou
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test timed out before producing results, preventing verification of support fraction or counterexamples. | next: Run test with extended timeout and debug crash logs

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	84923
2	preregistration	ollama_remote	qwen3:8b	0	0	19906
3	novelty	ollama_remote	qwen3:8b	0	0	16508
4	novelty	ollama_remote	qwen3:8b	0	0	12274
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10536
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8676
7	judge	ollama_remote	qwen3:8b	0	0	52030

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 204853 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/eff6dec1e004.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/eff6dec1e004.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/eff6dec1e004.tar.gz` (if generated)